ENTRY POINT
Starts the game
Manages main flow

«UI»
HelpScreen
-vector<Text> m_helpTexts
+draw(window)

ButtonType
START
HELP
EXIT
RESUME

«Core»
GameController
-WindowManager m_window
-GameMenu m_menu
-LevelManager m_levels
+run() : void

has

«UI»
GameMenu
-WindowManager& m_window
-vector<Button> m_buttons
+run() : ButtonType

THE CONDUCTOR
- Holds all game objects
- Runs game loop
- Updates physics
- Handles collisions
- Checks win/lose conditions

«Core»
LevelManager
-WindowManager& m_window
-FileManager m_fm
-LevelData m_levelData
-Level m_level
+run() : void

contains
uses
many

«UI»
Button
-RectangleShape m_shape
-Text m_text
-ButtonType m_type
+draw(window)
+contains(point) : bool
+getType() : ButtonType

«Core»
WindowManager
-RenderWindow m_window
+getWindow() : RenderWindow&
+pollEvent() : bool
+clear() : void
+display() : void
+draw() : void
+isOpen() : bool

«Core»
Level
-LevelData* m_levelData
-unique_ptr<Player> m_player
-vector<unique_ptr<MovingObject>> m_movableObj
-vector<vector<unique_ptr<StaticObject>>> m_staticObj
-Board m_board
-GameBar m_bar
-GameStatus m_gameStatus
-Clock m_clock
+run(window) : GameStatus
+setLevel(board, levelData) : void
-updateLevel() : void
-updateObject(deltaTime) : void
-handlColisions() : void
-drawLevel() : void
-checkPlayerDied() : void
-checkLevelComplete() : void
-resetLevel() : void

«Helper»
FileManager
-string m_fileName
+loadFromFile() : vector<string>
+saveToFile(boardData)
+getSizeBoard() : Vector2u

uses to create objects

«Singleton»
Factory
-map<ObjectSymbol, CreatorFunc> m_creators
-map<ObjectSymbol, int> m_counts
+instance() : Factory&
+creator(symbol, pos, data) : unique_ptr<Object>
+registerType(symbol, func) : bool
+resetCounts()

GameStatus
PLAYING
LEVEL_COMPLETE
GAME_OVER
QUIT

«creates»

«GameObject»
Object
#Sprite m_sprite
#Vector2f m_startPos
#ObjectSymbol m_symbol
#bool m_isActive
+update(deltaTime, levelData)
+handleCollision(Object&)
+handleCollision(Player&)
+handleCollision(Enemy&)
+handleCollision(Coin&)
+handleCollision(Wall&)
+handleCollision(Ground&)
+handleCollision(Ladder&)
+handleCollision(Rope&)
+reset()
+getPos() : Vector2f
+getBoundRec() : FloatRect
+isActive() : bool
+draw(window)

FACTORY PATTERN
Creates objects dynamically
based on ObjectSymbol
Example: '@' → Player

VISITOR PATTERN
Double dispatch for
collision handling
Each object knows how
to collide with every
other object type

«Singleton»
SoundManager
-Sound m_sound
+instance() : SoundManager&
+play(type: SoundType)
+stop()

«UI»
GameBar
-Text m_livesText
-Text m_scoreText
-Text m_timeText
-Text m_levelText
+draw(window, levelData)

contains (2D)
gets textures
uses
displays
has

«Singleton»
Resources
-Texture m_playerTexture
-Texture m_enemyTexture
-Texture m_wallTexture
-Texture m_coinTexture
-SoundBuffer m_coinBuffer
-Font m_damned
+instance() : Resources&
+loadResources()
+getTexture(symbol) : Texture&
+getSoundBuffer(type) : SoundBuffer&
+getFont(type) : Font&

«GameObject»
MovingObject
#Vector2f m_velocity
#Vector2f m_lastDir
#bool m_onLadder
#bool m_onRope
#bool m_onGround
#Board* m_board
+update(deltaTime, levelData)
+setBoradPtr(board: Board&)
#getDirection() : Vector2f
#move(deltaTime)
#isOnGround() : bool
#isOnLadder() : bool
#handleLadderLogic(ladder)
#handleRopeLogic(rope)
#resolveOverlap(obj)
#keepObjInsideMap()
#checkFalling(fallSpeed)
#getBoard() : Board*

uses (Player)
needs for movement

«Helper»
InputHandler
+isActionPressed(action) : bool
+getMovementDirection() : Vector2f
+isDig() : bool

PHYSICS ENGINE
- Gravity
- Velocity
- Ladder/Rope logic
- Collision resolution
- Needs Board for pathfinding

«Core»
Board
-vector<vector<unique_ptr<StaticObject>>>* m_map
-Player* m_player
-LevelData* m_levelData
+worldToGrid(pos) : Vector2i
+gridToWorld(pos) : Vector2f
+isWall(pos) : bool
+isLadder(pos) : bool
+tryToDig(digPos) : void
+getPlayerPosition() : Vector2f
+getSymbolAt(index) : ObjectSymbol
+isValidIndex(index) : bool

HELPER FOR LEVEL
- Grid ↔ World conversion
- Pathfinding queries
- Collision checks
- Digging logic

ObjectSymbol
PLAYER = '@'
ENEMY = '%'
COIN = '*'
WALL = '#'
GROUND = '^'
LADDER = 'H'
ROPE = '-'

SINGLETON
Loads all assets once
Provides global access
to textures & sounds

SoundType
COIN
LIFE_LOST
FALL
LEVEL_COMPLETE

references

«Data»
LevelData
+int currentLevel
+int score
+int lives
+float timeLeft
+int coinsLeft
+bool levelComplete
+bool playerDied
+addScore(value)
+loseLife()
+updateTime(dt)

«GameObject»
Enemy
+getDirection() : Vector2f

«GameObject»
SmartEnemy
+update(deltaTime, levelData)
+getDirection() : Vector2f
+handleCollision(...)
-getSmartNextDir(start, target) : Vector2f

«GameObject»
DumbEnemy
-float m_changeDirTimer
-Vector2f m_currentDir
+update(deltaTime, levelData)
+getDirection() : Vector2f
+handleCollision(...)

RANDOM AI
Changes direction
randomly every few seconds

BFS ALGORITHM
Uses Board to find
shortest path to Player
Avoids walls and obstacles

Action
MOVE_UP
MOVE_DOWN
MOVE_LEFT
MOVE_RIGHT
DIG_LEFT
DIG_RIGHT

«GameObject»
Player
-PlayerPackage m_package
-DeathReason m_deadReason
+update(deltaTime, levelData)
+getDirection() : Vector2f
+handleCollision(...)
+playerKilled()
+lifeLost()
+getPackage() : PlayerPackage&
#takeCoin(value)
#digGround()

«GameObject»
StaticObject
+update(deltaTime, levelData)
+reset()

«Data»
PlayerPackage
+int lives
+int score
+int level
+addLife()
+loseLife()
+addScore(value)

DeathReason
NONE
ENEMY
FALL
TIMEOUT
SMASHED

«GameObject»
Ground
-bool m_isDug
-Clock m_digTimer
+dig() : void
+isDug() : bool
+handleCollision(...)

«GameObject»
Ladder
+handleCollision(...)

«GameObject»
Rope
+handleCollision(...)

«GameObject»
Coin
-int m_value
+getValue() : int
+handleCollision(...)

«GameObject»
Wall
+handleCollision(...)